1495 UNIVERSITY OF ABERDEEN

JC4004 – Computational Intelligence

Sessions 25 & 26: Advanced convolutions

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)
Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)
Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

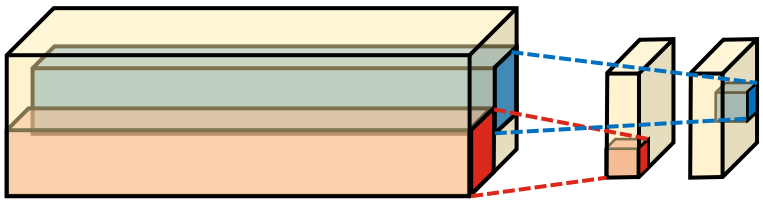
Oct-Dec 2025

Advanced convolutions

- In addition to the basic 2D convolution, there is a range of variants of convolution useful for different kinds of applications
- We introduce some common advanced convolution variants:
  - *Grouped convolutions* for making the models smaller
  - *Dilated convolutions* widening the perceptive field
  - *3D convolutions* for 3D input data
  - *Transposed convolution* for upsampling, and U-Net architecture employing transposed convolutions

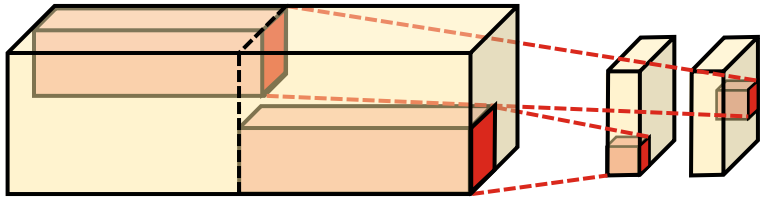
Grouped convolutions

- In full 2D convolution, convolution kernels have size $H \times W \times C$, where C is the number of input channels
- One kernel for each output channel: a lot of learnable weights required!
  - One way to reduce the number of learnables is to use *grouped convolutions* where the input channels are divided into groups



full 2D convolution

*C*1 *C*2

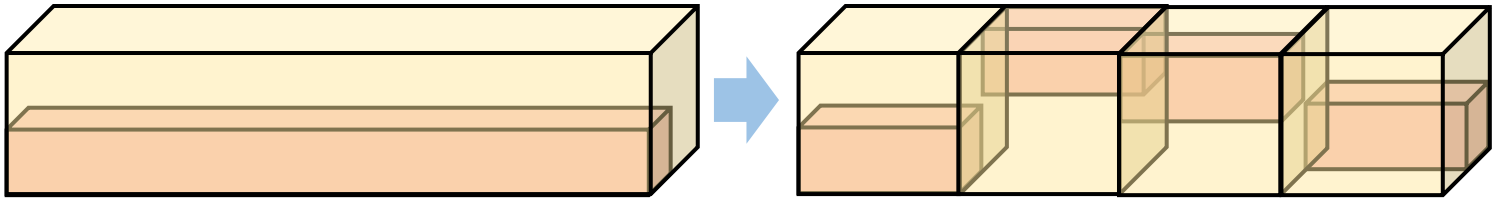


grouped convolution

*C*1 *C*2

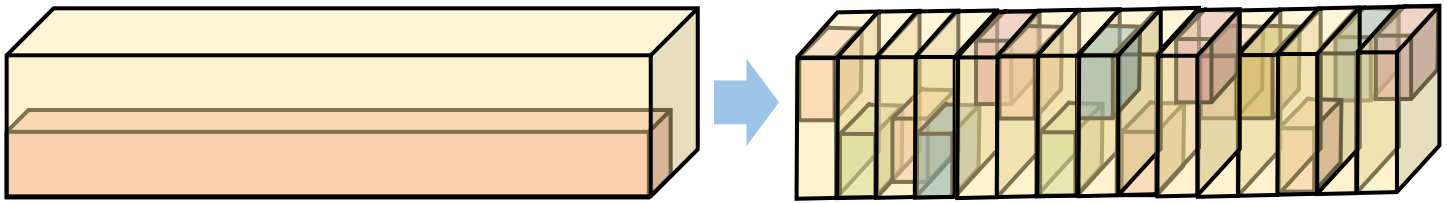
Grouped convolution definition

- In grouped convolutions, the input channel dimension is divided into groups and convolution operations applied to each group separately
  - The same set of convolution kernels can be used for all the groups
  - ***Example:*** for 16 input channels and 32 output channels, full convolution requires 32 kernels of size $H \times W \times 16$; if we use grouped convolution with four groups, we only need 8 kernels of size $H \times W \times 4$



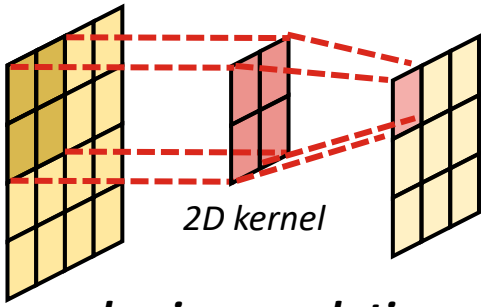
Depthwise convolution

- In depthwise convolution, each channel is convolved using its own genuinely 2-dimensional convolution kernel
  - Can be considered as a special case of grouped convolution where each channel forms its own group, and all the channels use different kernels
  - Efficient computation, but ignores interactions between channels

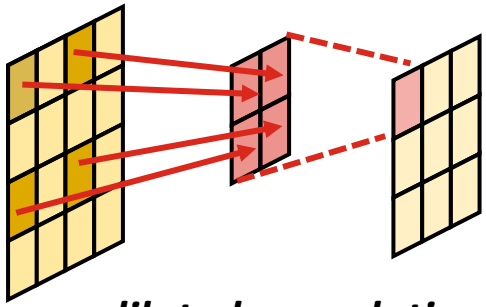


Dilated convolution

- To keep models reasonable in size, spatial dimensions of convolution kernels cannot be very large
  - One convolution operation only covers small area, or *receptive field*
  - *Dilated convolution* can be used to expand the receptive field without increasing the kernel size



basic convolution



dilated convolution

Definition of dilated convolution

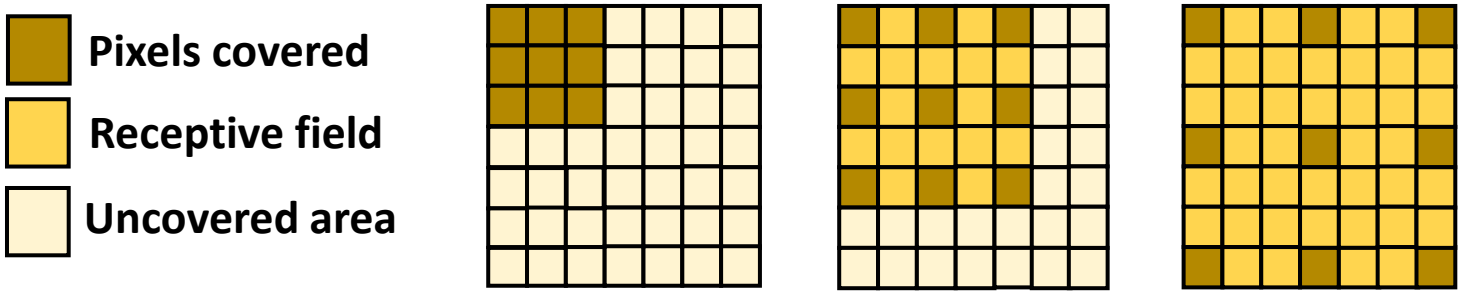
- Given input image $x(i,j)$ where i and j are the coordinates of a pixel, convolution kernel K of height h_K and width w_K, and dilation factor d, the output image pixel $y(i,j)$ in position i,j can be computed with:

$$y(i,j) = \sum_{m=0}^{h_K-1} \sum_{n=0}^{w_K-1} K(m,n) \cdot x(i+m \cdot d, j+n \cdot d)$$

- For simplicity, channel dimension is ignored, stride is assumed to be 1, and overflow across image borders is not considered in the equation

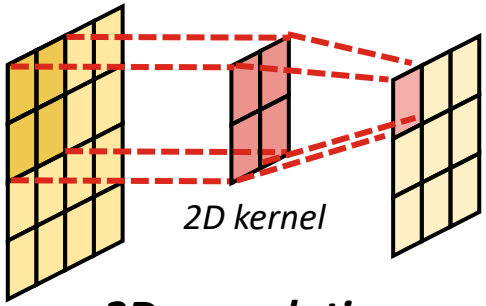
Characteristics of dilated convolution

- Dilated convolution operation skips part of the pixels
  - However, if stride is 1 and dilation factor is 2, all the pixels are covered
  - Dilated convolution could remove the need for pooling layers
  - Dilated convolution not optimal for extracting very fine details

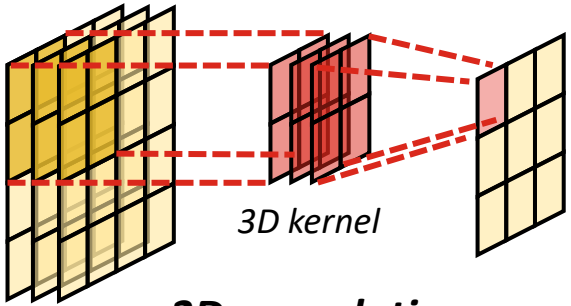


3D convolution

- Traditional 2D convolution operates on two spatial dimensions
  - The concept can be extended to 3D by using 3D convolution kernels
  - The third dimension can represent e.g., time, as in video sequences
  - The kernel can slide in three dimensions to produce 3D output



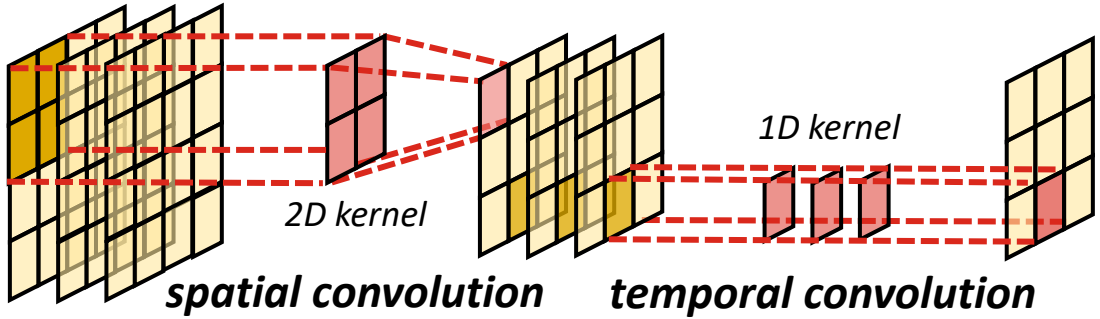
2D convolution



3D convolution

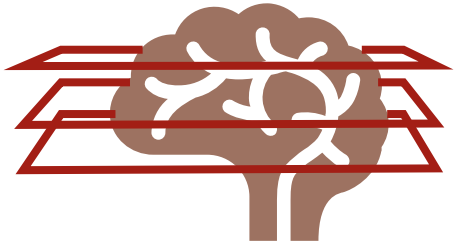
Factorised convolution

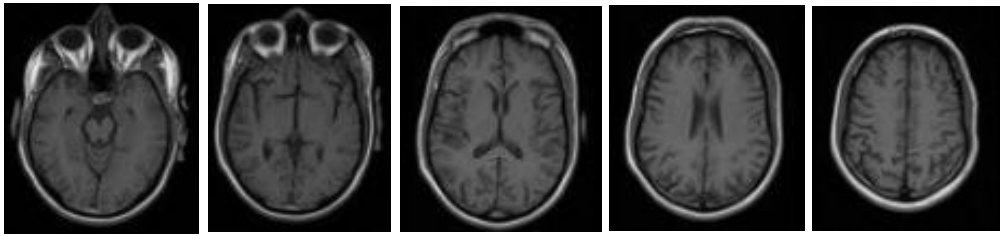
- 3D convolution kernel size is $H \times W \times T \times C$, where H, W, T, and C represent the kernel height, width, time, and channel dimensions
  - The number of learnable parameters grows excessively along dimensions
  - More efficient solution is *factorised (2+1)D convolution* where spatial 2D and temporal 1D convolutions are performed separately



Applications of 3D convolution

- 3D convolution and (2+1)D convolution can be used on any 3D input
  - The most obvious application is video classification
  - In medical imaging, 3D magnetic resonance images (MRI) are often composed from 2D slices, and they can be analysed with 3D CNNs

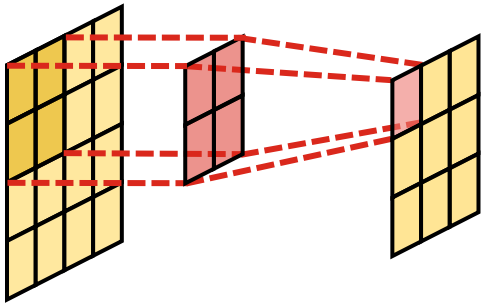




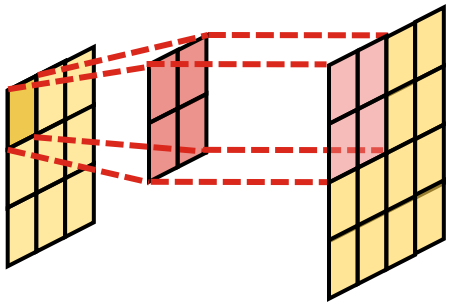
Brain images cropped from: Turken et al., "Multimodal surface-based morphometry reveals diffuse cortical atrophy in traumatic brain injury," BMC Medical Imaging 9(1):20, December 2009.

Transposed convolution

- Traditional convolution operation reduces the spatial resolution
  - Information is lost, so the convolution operation cannot be inversed
  - However, we can define *transposed convolution* (sometimes misleadingly referred to as *inverse convolution* or *deconvolution*) for upsampling



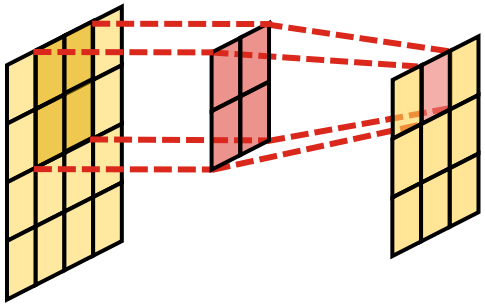
convolution



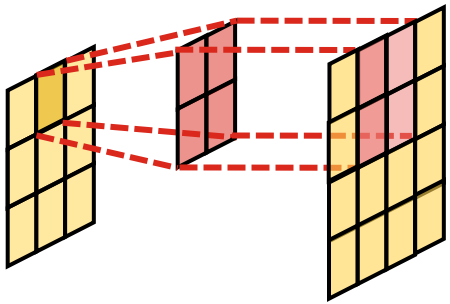
transposed convolution

Transposed convolution

- Traditional convolution operation reduces the spatial resolution
  - Information is lost, so the convolution operation cannot be inversed
  - However, we can define *transposed convolution* (sometimes misleadingly referred to as *inverse convolution* or *deconvolution*) for upsampling



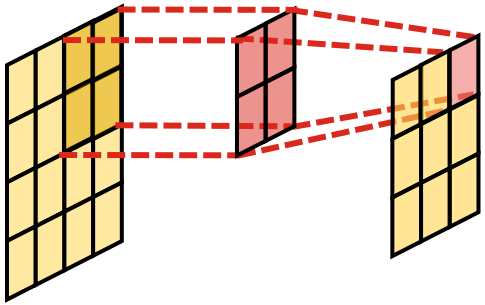
convolution



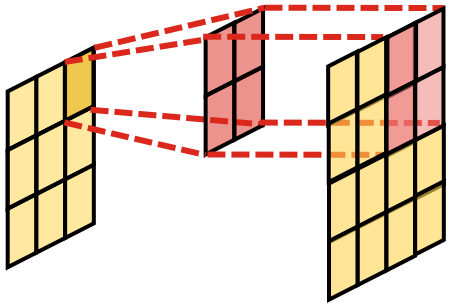
transposed convolution

Transposed convolution

- Traditional convolution operation reduces the spatial resolution
  - Information is lost, so the convolution operation cannot be inversed
  - However, we can define *transposed convolution* (sometimes misleadingly referred to as *inverse convolution* or *deconvolution*) for upsampling



convolution



transposed convolution

Definition of transposed convolution

- Given input image $x(i,j)$ where i and j are the coordinates of a pixel, and convolution kernel K of height h_K and width w_K, the output image pixel $y(i,j)$ in position i,j can be computed with:

$$y(i,j) = \sum_{m=0}^{h_K-1} \sum_{n=0}^{w_K-1} K(m,n) \cdot x(i-m, j-n)$$

- For simplicity, channel dimension is ignored, stride is assumed to be 1, and overflow across image borders is not considered in the equation

Definition of transposed convolution

- Given input image $x(i,j)$ where i and j are the coordinates of a pixel, and convolution kernel K of [illegible] age pixel $y(i,j)$ in position i,j ca[illegible]

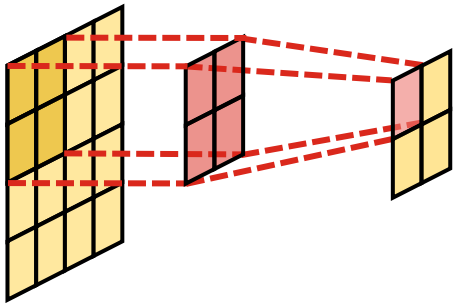
Here, $x(i,j) = 0$,
if $i < 0$ or $j < 0$ or $i \geq H$ or $j \geq W$

$$y(i,j) = \sum_{m=0}^{h_K-1} \sum_{n=0}^{w_K-1} K(m,n) \cdot x(i-m, j-n)$$

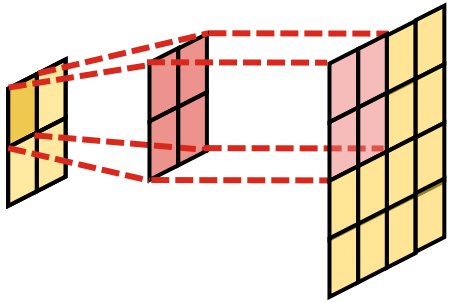
- For simplicity, channel dimension is ignored, stride is assumed to be 1, and overflow across image borders is not considered in the equation

Upsampling with transposed convolution

- Transposed convolution can be used for upsampling by using a stride that is larger than one
  - Note that unlike in normal convolution, the stride is applied to the output



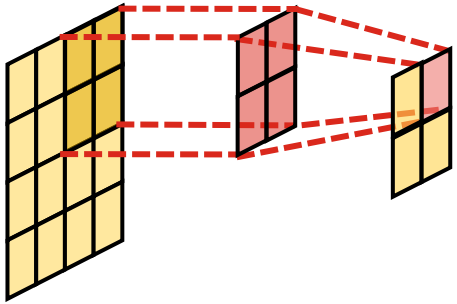
Convolution (s=2)



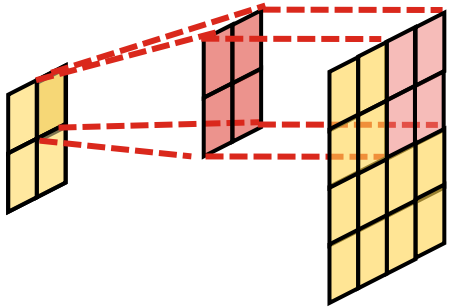
transposed convolution (s=2)

Upsampling with transposed convolution

- Transposed convolution can be used for upsampling by using a stride that is larger than one
  - Note that unlike in normal convolution, the stride is applied to the output



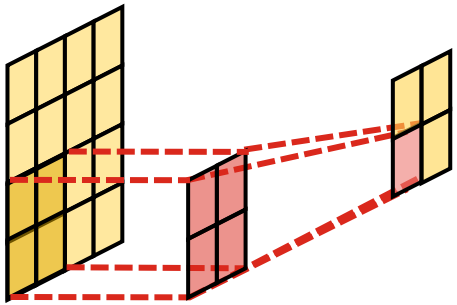
Convolution (s=2)



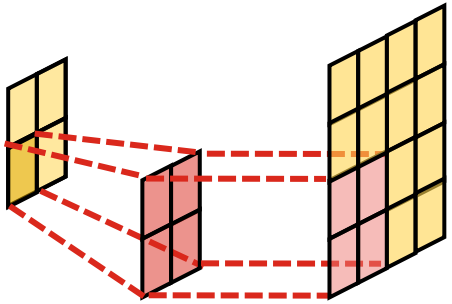
transposed convolution (s=2)

Upsampling with transposed convolution

- Transposed convolution can be used for upsampling by using a stride that is larger than one
  - Note that unlike in normal convolution, the stride is applied to the output



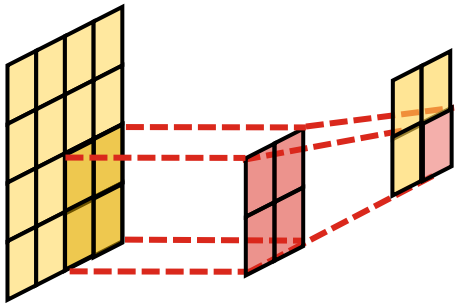
Convolution (s=2)



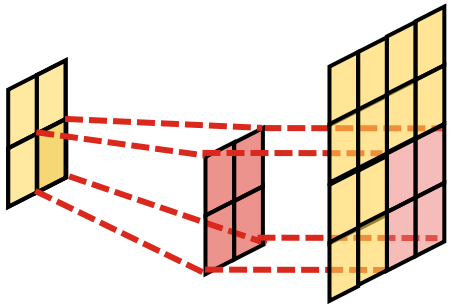
transposed convolution (s=2)

Upsampling with transposed convolution

- Transposed convolution can be used for upsampling by using a stride that is larger than one
  - Note that unlike in normal convolution, the stride is applied to the output



Convolution (s=2)



transposed convolution (s=2)

Output size of transposed convolution

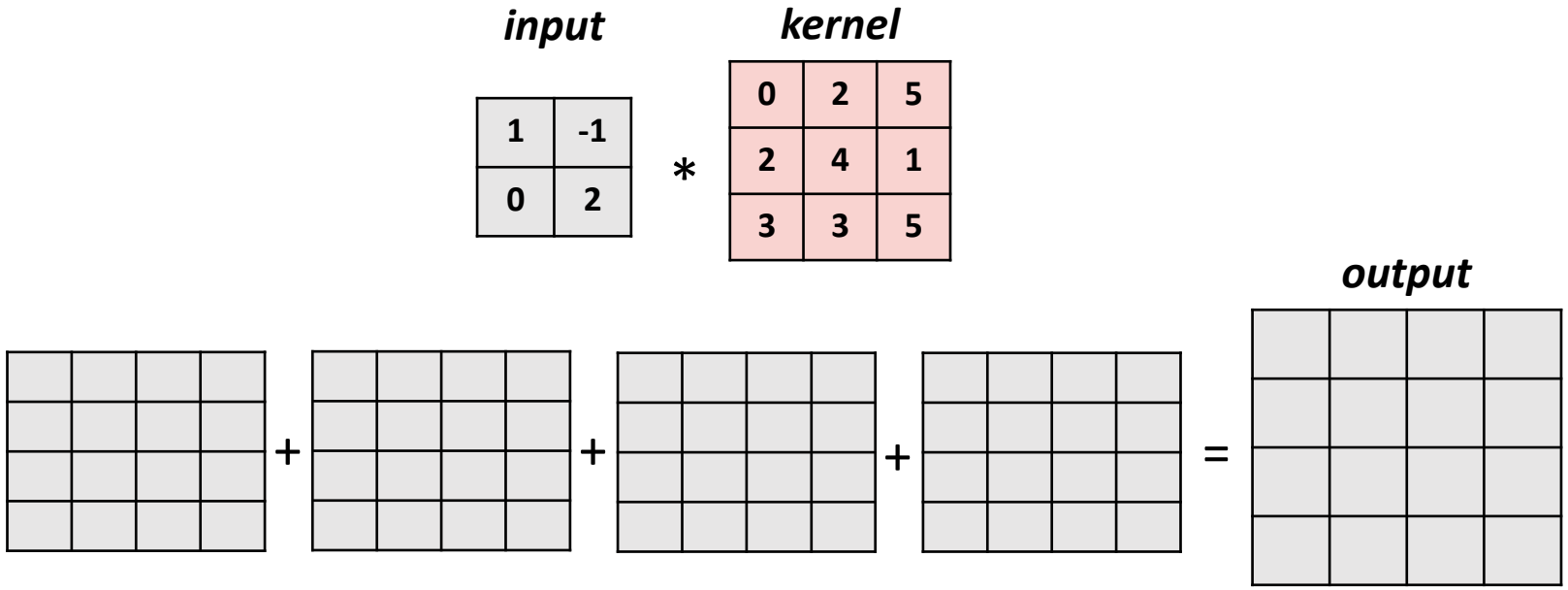
- Transposed convolution output width w_y and height h_y are solved as:

$$w_y = (w_x - 1)s + w_K - 2p_x + p_y$$
$$h_y = (h_x - 1)s + h_K - 2p_x + p_y$$

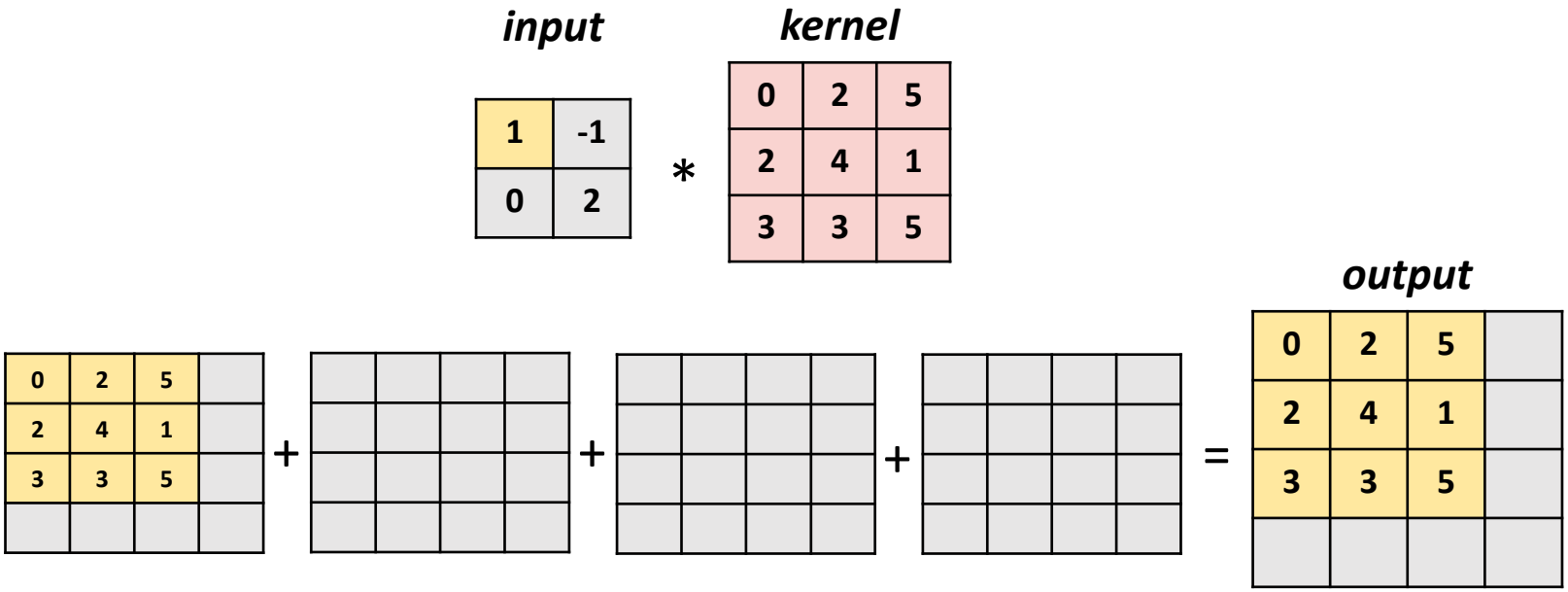
- Here, w_x and h_x are the width and height of the input image, w_K and h_K are the width and height of the kernel, s is the stride, and p_x and p_y are the input and output paddings, respectively

Break: any questions?

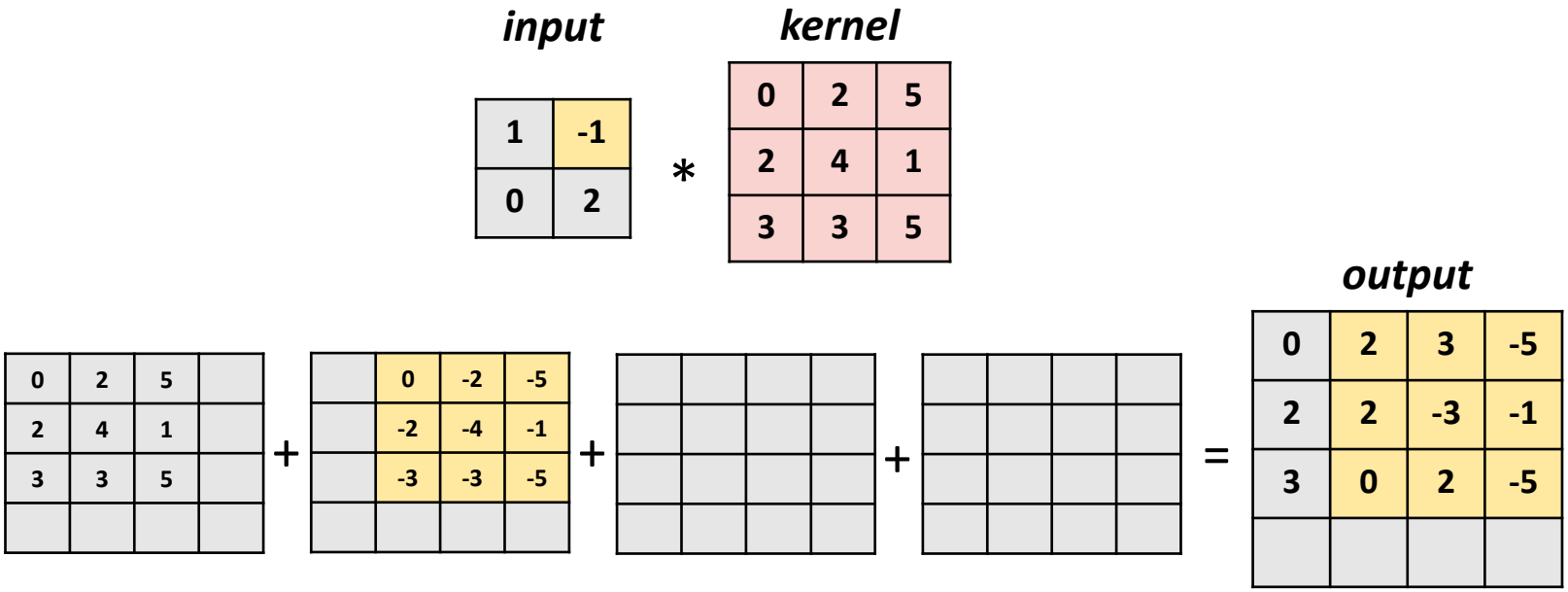
Transposed convolution example



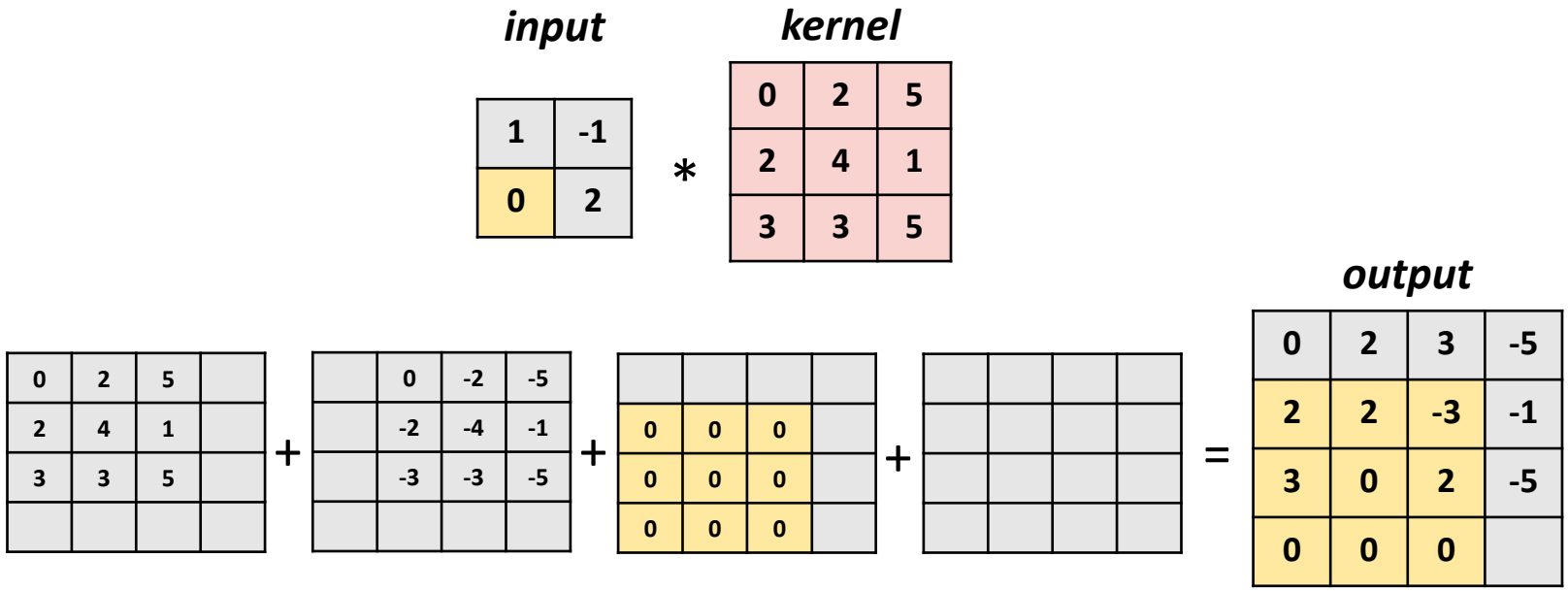
Transposed convolution example



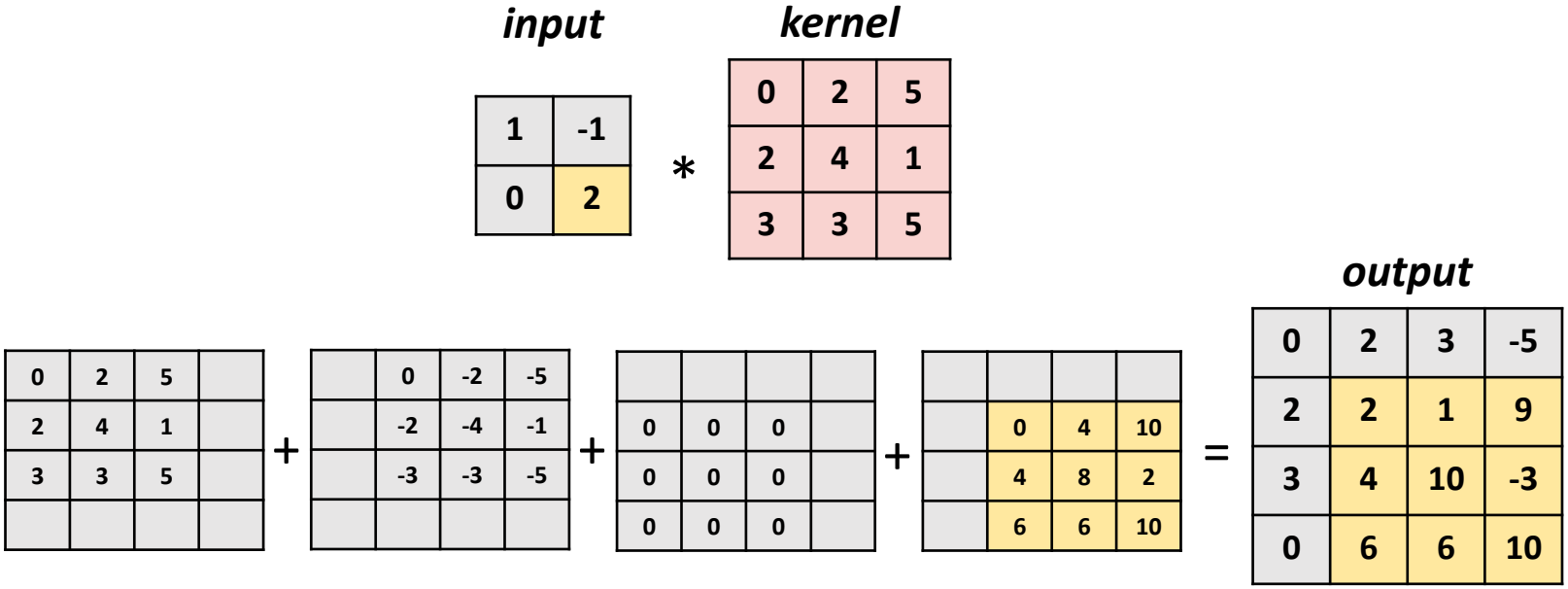
Transposed convolution example



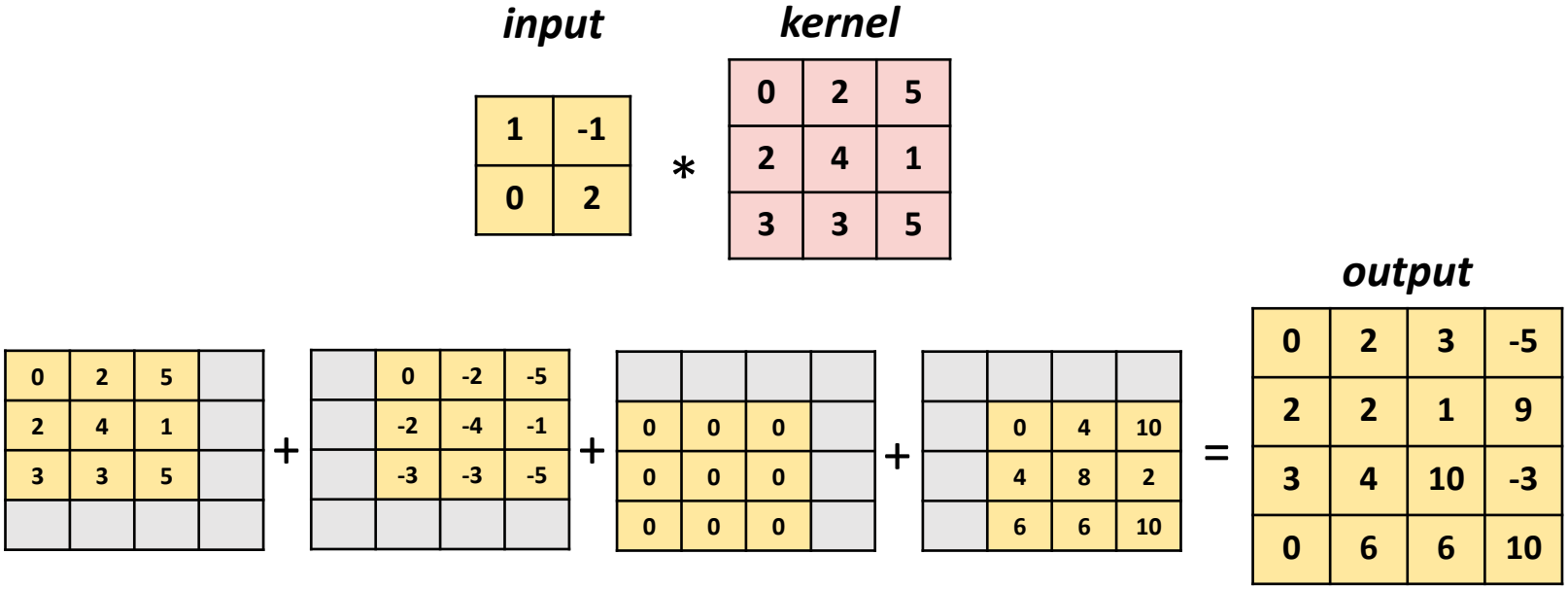
Transposed convolution example



Transposed convolution example



Transposed convolution example



Example Python implementation

- The following Python code implements transposed convolution

```
1   import torch
2
3   def trans_conv(X, K):
4       h, w = K.shape
5       Y = torch.zeros((X.shape[0]+h-1, X.shape[1]+w-1))
6       for i in range(X.shape[0]):
7           for j in range(X.shape[1]):
8               Y[i: i+h, j: j+w] += X[i,j] * K
9
10      return Y
```

Example Python implementation

- We can test it using the previously shown example

```
1    import torch
2
3    def trans_conv(X, K):
...  ...
11
12   X = torch.tensor([[1,-1],[0,2]])
13   K = torch.tensor([[0,2,5],[2,4,1],[3,3,5]])
14   print(trans_conv(X,K))
```

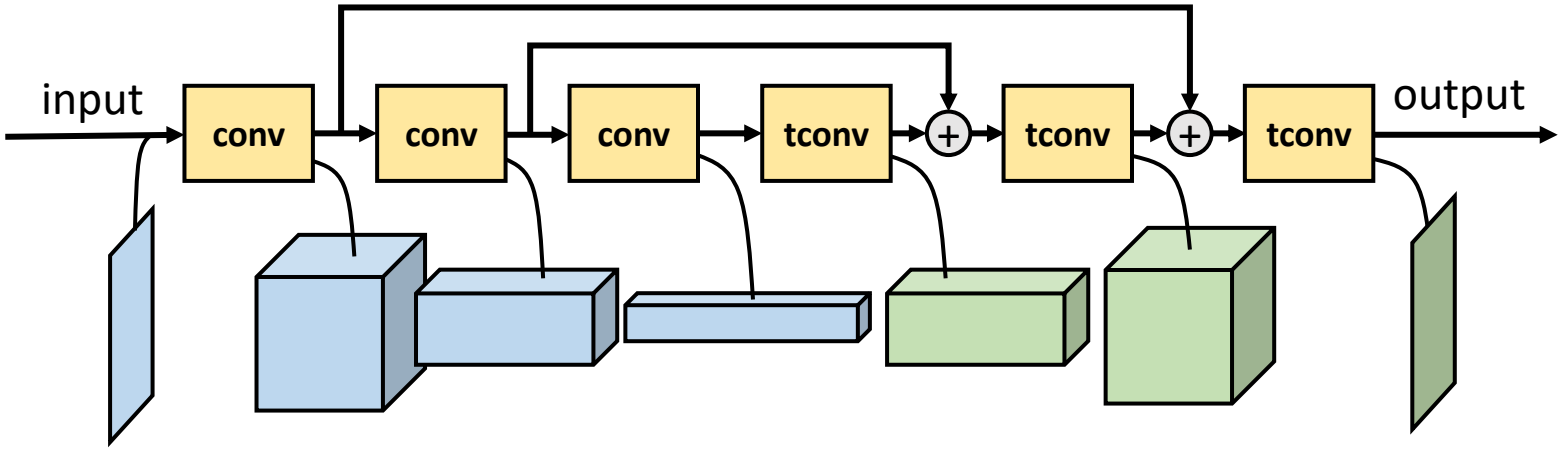
```
tensor([[ 0.,  2.,  3., -5.],
        [ 2.,  2.,  1.,  9.],
        [ 3.,  4., 10., -3.],
        [ 0.,  6.,  6., 10.]])
```

Introduction to U-Nets

- *U-Net* is a convolutional neural network where the traditional convolution layers are succeeded by upsampling layers
  - U-Net can produce an output with the same or even larger spatial resolution than the original input
  - U-Nets were designed originally for image segmentation, but they also have other applications, such as image denoising
- U-Nets typically use transposed convolution as an upsampling operator
  - A variant of convolution that can produce output in a higher resolution than the input

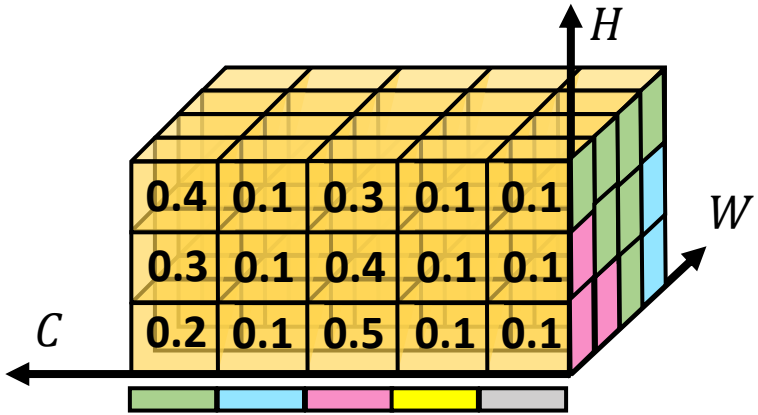
U-Net architecture

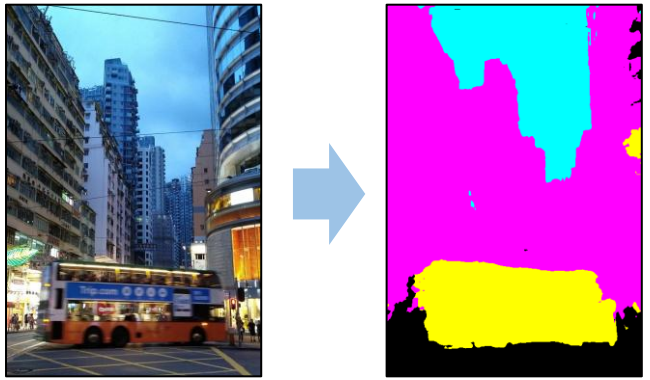
- In typical U-Net architectures, there is a traditional convolutional network part followed by symmetric upsampling part
  - Respective layers usually connected with skip connections



U-Nets for image segmentation

- U-Nets were originally developed for semantic image segmentation
  - The output has dimensions $H \times W \times C$, where height H and width W are the same as the input of the image, and the channel dimension C represents the distribution for predefined semantic classes



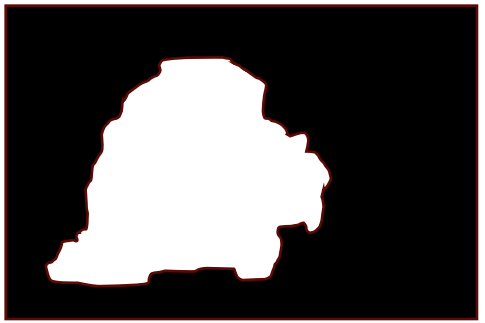


Loss functions for segmentation

- Loss functions typically compare segmentation masks with ground truth
  - For each class, pixels that belong to a class are 1 and pixels that do not belong to a class are 0
  - Two common loss functions: binary cross-entropy and DICE loss

Original image

Ground truth mask



Predicted mask

Binary cross-entropy

- Binary cross-entropy is computed for each of the N masks:

$$L_{BCE} = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(p(y_i)) + (1 - y_i)\log(1 - p(y_i))$$

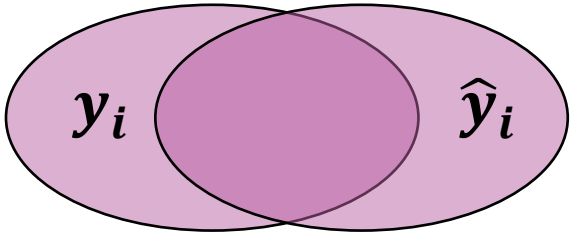
  - y_i is the ground truth (0 or 1) and $p(y_i)$ is the predicted probability for y_i
  - Binary cross entropy does not take the imbalance between classes into account, so often weighted cross-entropy is used instead:

$$L_{BCE} = -\frac{1}{N}\sum_{i=1}^{N} \beta y_i \log(p(y_i)) + (1 - \beta)(1 - y_i)\log(1 - p(y_i)); \;\; \beta = \frac{n_{y=0}}{n_{total}}$$

DICE loss

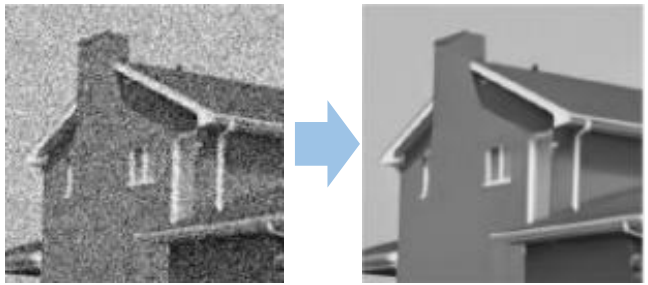
- DICE loss (or DICE coefficient) is another commonly used loss function based on the number of pixels intersecting in the predicted and ground truth masks:

$$L_{DICE} = \frac{1}{N}\sum_{i=1}^{N} 1 - \frac{2y_i\hat{y}_i + 1}{y_i + \hat{y}_i + 1}$$



U-Nets for image-to-image translation

- U-Nets can be used for translating image to another image
  - Image denoising and other filtering tasks
  - Converting a photographic image to a cartoon image or vice versa
  - Adding colours to grayscale images (e.g., historic photos)
  - etc…



From: Jia et al., "DDUNet: Dense Dense U-Net with Applications in Image Denoising," ICCV 2021

From: Zoyd et al., "Image Colorization using U-Net with Skip Connections and Fusion Layer on Landscape Images," arXiv:2205.12867

Summary

- Advanced variants of convolution can be used in applications where the basic 2D convolution is not suitable
- Some of the common advanced convolution variants outlined
  - Grouped convolution: helps to reduce model size
  - Dilated convolution: increases receptive field
  - 3D convolution: straightforward extension of 2D convolution to three dimensions; can be used in e.g., video or medical image analysis
  - Transposed convolution often used for upsampling in U-Nets
  - U-Net architecture useful in many applications, such as image segmentation, denoising, colouring, etc.

Questions and discussion